



本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 编译内核利用已有内核构建 OS

专业名称: 计算机科学与技术 (人工智能与大数据方向)

学生姓名: 陶昕璨

学生学号: 24325242

实验地点: 实验中心 D501

实验时间: 2026/3/9-2026/3/12

目录

一、实验概述·····	1
二、预备知识与实验环境·····	1
三、实验任务·····	1
四、实验步骤与实验结果·····	1
五、实验总结与心得体会·····	20
六、附录：参考资料清单·····	20

Section 1 实验概述

学习到 x86 汇编、计算机的启动过程、IA-32 处理器架构和字符显存原理。根据所学的知识，能自己编写程序，然后让计算机在启动后加载运行，以此增进对计算机启动过程的理解，为后面编写操作系统加载程序奠定基础。同时，将学习如何使用 gdb 来调试程序的基本方法

Section 2 预备知识与实验环境

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具、计算机启动原理、IA-32 处理器架构、GDB 调试基础、C++程序运行
- 实验环境：
 - 虚拟机版本/处理器型号：openSUSE（64-bit）
 - 代码编辑环境：Vim（终端）/ Gedit（图形化）
 - 代码编译工具：nasm、qemu-system-i386、GDB
 - 重要三方库信息：libc6-dev-i386、gcc-multilib、qemu-utils

Section 3 实验任务

- 实验任务 1：学习 IA-32 处理器体系结构
- 实验任务 2：学习汇编语言
- 实验任务 3：情动 os 并在屏幕输出蓝色的 Hello World
- 实验任务 4：使用 GDB 对 MBR 的代码进行 debug
- 实验任务 5：完成思考题 14、16、17

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：学习 IA-32 处理器体系结构，了解实模式
- 实验步骤：

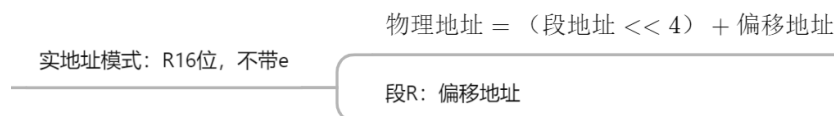
①汇编语言的学习

在编写操作系统的过程中，我们需要用到一些特权指令，如读写硬件端口、关/开中断等。而高级语言并未提供相应的指令，只有汇编语言才提供。

②IA-32 处理器体系结构的学习

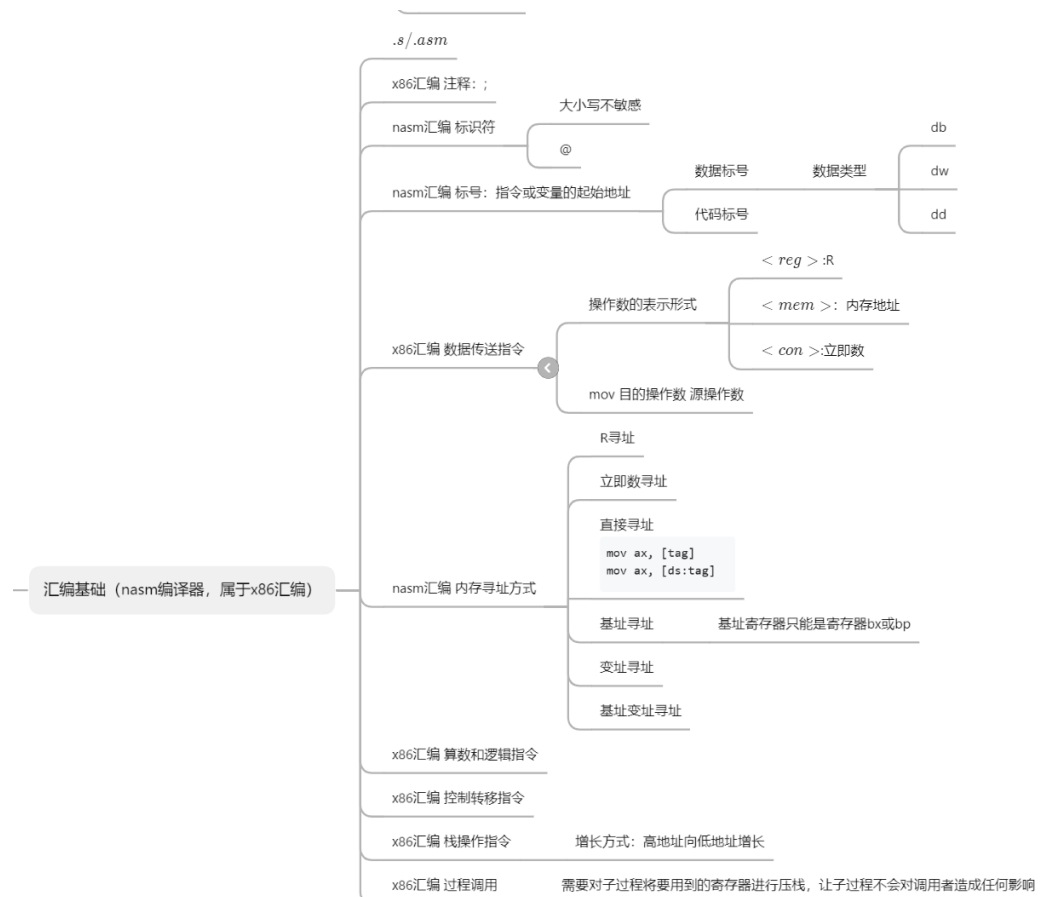


③实地址模式



----- 实验任务 2 -----

- 任务要求: 汇编基础 (nasm 编译器, 属于 x86 汇编)
- 实验步骤:



实验任务 3

- 任务要求: os 启动 Hello World

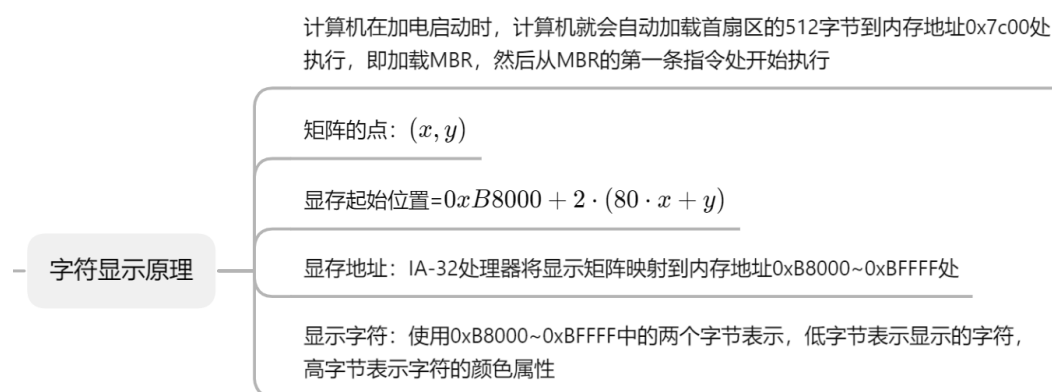
- 思路分析:

1. 注意显存地址的计算

2. 段寄存器不能直接赋值立即数, 需借助通用寄存器

- 实验步骤:

① 学习字符显示原理



②使用 nano（命令行文本编辑器）创建并编辑 mbr.asm 文件

```
lala@lala-VirtualBox: ~/mbr_experiment$ nano mbr.asm
```

③编写 mbr 代码



```
org 0x7c00
[bits 16]
xor ax, ax ; eax = 0
; 初始化段寄存器，段地址全部设为0
mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax

; 初始化栈指针
mov sp, 0x7c00
mov ax, 0xb800
mov gs, ax

mov ah, 0x01 ; 青色
mov al, 'H'
mov [gs:2 * 0], ax
```

④编译生成二进制文件

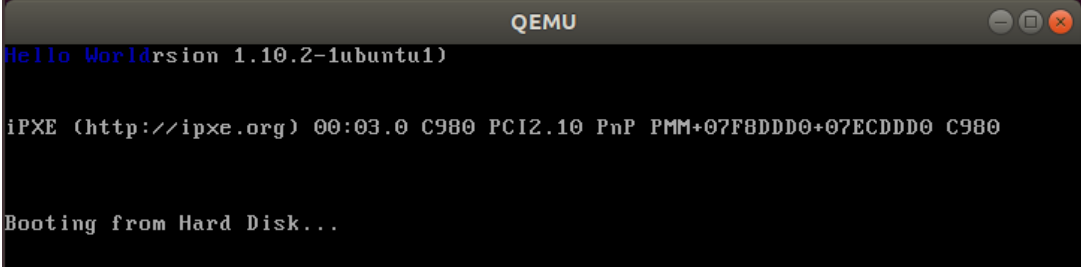
```
lala@lala-VirtualBox: ~/mbr_experiment$ nasm -f bin mbr.asm -o mbr.bin
lala@lala-VirtualBox: ~/mbr_experiment$ ls
mbr.asm  mbr.bin
```

⑤创建 10MB 的虚拟硬盘文件并且将 MBR 写入其首扇区★

```
lala@lala-VirtualBox: ~/mbr_experiment$ qemu-img create hd.img 10m
Formatting 'hd.img', fmt=raw size=10485760
lala@lala-VirtualBox: ~/mbr_experiment$ dd if=mbr.bin of=hd.img bs=512 count=
1 seek=0 conv=notrunc
记录了1+0 的读入
记录了1+0 的写出
512 bytes copied, 9.6347e-05 s, 5.3 MB/s
lala@lala-VirtualBox: ~/mbr_experiment$ ls
hd.img  mbr.asm  mbr.bin
```

⑥启动 qemu 模拟计算机启动

```
lala@lala-VirtualBox:~/mbr_experiment$ qemu-system-i386 -hda hd.img -serial null -parallel stdio
```



The QEMU window displays the following text:

```
Hello World (rsion 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980

Booting from Hard Disk...
```

----- 实验任务 4 -----

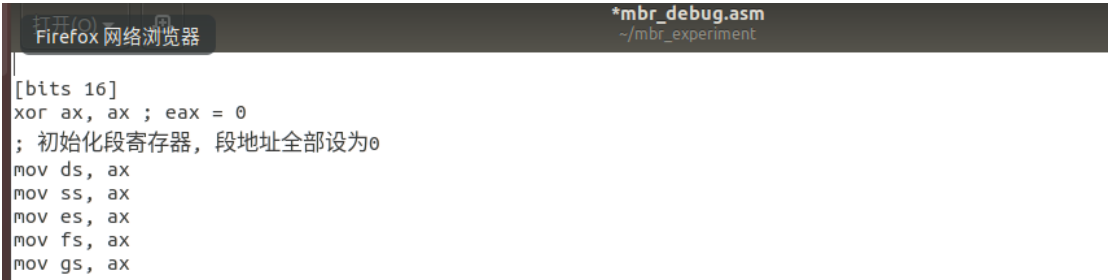
- 任务要求：使用 gdb 配合 qemu 进行 debug
- 思路分析：

1. gdb 调试生成符号表时需删除 org 伪指令

2. QEMU 加-s -S 开启调试

- 实验步骤：

①删除伪指令



The screenshot shows the following assembly code in a file named `mbr_debug.asm`:

```
[bits 16]
xor ax, ax ; eax = 0
; 初始化段寄存器, 段地址全部设为0
mov ds, ax
mov ss, ax
mov es, ax
mov fs, ax
mov gs, ax
```

②生成符号表

```
lala@lala-VirtualBox:~/mbr_experiment$ nasm -o mbr.o -g -f elf32 mbr_debug.asm
lala@lala-VirtualBox:~/mbr_experiment$ ld -o mbr.symbol -melf_i386 -N mbr.o -Ttext 0x7c00
ld: 警告: 无法找到项目符号 _start; 缺省为 0000000000007c00
lala@lala-VirtualBox:~/mbr_experiment$
```

③打开 GDB 执行并调试命令

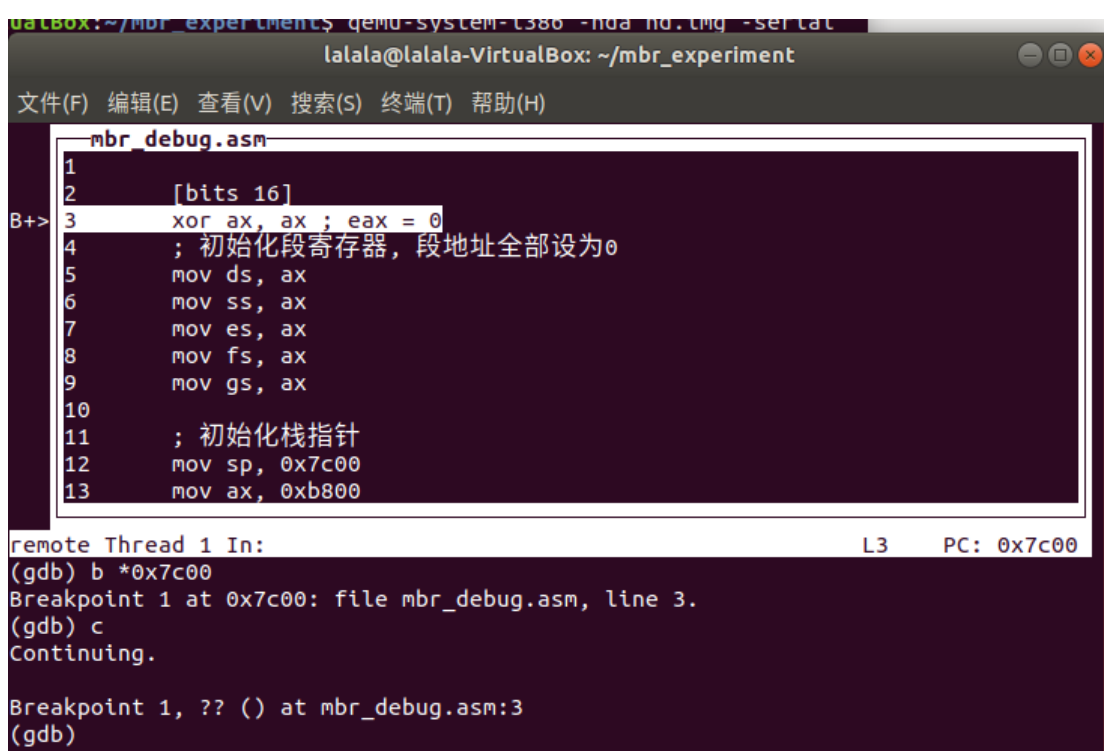
▲准备操作，打开可视化调试

```
# 1. 连接到QEMU的调试端口（默认1234）
target remote:1234

# 2. 加载符号表，指定代码起始地址为0x7c00
add-symbol-file mbr.symbol 0x7c00

# 3. 打开源代码显示窗口（可视化调试）
layout src
```

▲设置断点，执行到断点处



The screenshot shows a GDB terminal window titled "lalala@lalala-VirtualBox: ~/mbr_experiment". The window contains a list of assembly instructions from "mbr_debug.asm". Line 3, "xor ax, ax ; eax = 0", is highlighted. Below the assembly list, the GDB prompt shows the command "(gdb) b *0x7c00" and the response "Breakpoint 1 at 0x7c00: file mbr_debug.asm, line 3." followed by "(gdb) c" and "Continuing." The status bar at the bottom indicates "remote Thread 1 In: L3 PC: 0x7c00".

```
remote Thread 1 In: L3 PC: 0x7c00
(gdb) b *0x7c00
Breakpoint 1 at 0x7c00: file mbr_debug.asm, line 3.
(gdb) c
Continuing.

Breakpoint 1, ?? () at mbr_debug.asm:3
(gdb)
```

▲单步执行

```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
mbr_debug.asm
1
2     [bits 16]
3     xor ax, ax ; eax = 0
4     ; 初始化段寄存器, 段地址全部设为0
5     mov ds, ax
6     mov ss, ax
7     mov es, ax
8     mov fs, ax
9     mov gs, ax
10
11    ; 初始化栈指针
12    mov sp, 0x7c00
13    mov ax, 0xb800

Remote Thread 1 In: L5 PC: 0x7c02
(gdb) b *0x7c00
Breakpoint 1 at 0x7c00: file mbr_debug.asm, line 3.
(gdb) c
Continuing.

Breakpoint 1, ?? () at mbr_debug.asm:3
(gdb) n
(gdb)
```

④查看执行结果

```
QEMU - Press Ctrl-Alt to exit mouse grab
Hello Worldrsion 1.10.2-1ubuntu1)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECD000 C980

Booting from Hard Disk...
```

----- 思考题 16 -----

- 任务要求：实模式中断

- 实验步骤：

▲探索实模式下的光标中断，利用中断实现光标的位置获取和光标的移动

①学习关于 int 10h 实模式中断，理解各个功能号的用途

功能	功能号	参数	返回值
设置光标位置	AH=02H	BH=页码, DH=行, DL=列	无
获取光标位置和形状	AH=03H	BX=页码	AX=0, CH=行扫描开始, CL=行扫描结束, DH=行, DL=列
在当前光标位置写字符和属性	AH=09H	AL=字符, BH=页码, BL=颜色, CX=输出字符的个数	无

②编写获取初始光标位置（AH=03H）汇编代码

```

;获取初始光标位置
mov ah, 03H
mov bx, 0
int 10h

; 打印初始坐标
mov ax, 0xb800
mov gs, ax
mov ah, 02 ; 绿色属性
mov al, '('
mov [gs:2*0], ax
mov al, dh
mov [gs:2*1], ax
mov al, ','
mov [gs:2*2], ax
mov al, dl
mov [gs:2*3], ax
mov al, ')'
mov [gs:2*4], ax

```

③移动光标到第 1 行第 5 列（AH=02H），并打印 DH, DL

```

; 移动光标到第1行第5列
mov ah, 02H
mov bh, 0
mov dh, 1
mov dl, 5
int 10h

```

④获取移动后光标位置

```
; 验证移动后光标位置并打印
mov ah, 03H
mov bx, 0
int 10h
mov ah, 0x04    ; 红色属性
mov al, '('
mov [gs:2*160 + 2*0], ax ; |
mov al, dh
add al, 0x30
mov [gs:320 + 2*1], ax
mov al, ','
mov [gs:320 + 2*2], ax
mov al, dl
add al, 0x30
mov [gs:320 + 2*3], ax
mov al, ')'
mov [gs:320 + 2*4], ax
```

▲修改 Hello World 的代码，使用实模式下的中断来输出学号

①设置颜色：黄色更明显

```
mov bh, 0
mov bl, 0x0E    ; 黄
mov cx, 1
```

②依次在当前光标位置写字符（AH=09H）来打印学号 24325242

<pre> ; 输出数字2 mov ah, 09H mov al, '2' int 10h ; 光标右移1列 mov ah, 02H inc dl int 10h ; 输出数字4 mov ah, 09H mov al, '4' int 10h mov ah, 02H inc dl int 10h ; 输出数字3 mov ah, 09H mov al, '3' int 10h mov ah, 02H inc dl int 10h ; 输出数字2 mov ah, 09H mov al, '2' int 10h </pre>	<pre> ; 输出数字5 mov ah, 09H mov al, '5' int 10h mov ah, 02H inc dl int 10h ; 输出数字2 mov ah, 09H mov al, '2' int 10h mov ah, 02H inc dl int 10h ; 输出数字4 mov ah, 09H mov al, '4' int 10h mov ah, 02H inc dl int 10h ; 输出数字2 mov ah, 09H mov al, '2' int 10h </pre>
---	---

▲探索实模式的键盘中断，利用键盘中断实现键盘输入（int 16h AH=00H）并回显(int 10h AH=09H)，使用扫描码实现退出

(1) 0号功能调用 格式：MOV AH, 0

INT 16H

功能：从键盘读入字符送AL寄存器。执行时，等待键盘输入，一旦输入，字符的ASCII码放入AL中。

若AL = 0，则AH为输入的扩展码。

(esc键的ascii码十进制表示为27，二进制表示为0001 1011，十六进制表示为1B) cmp al, 1bh ;

可以判断刚才输入的字符是否是escape

```

keyboard_loop:
    ; 等待键盘输入 (int 16h AH=00H)
    mov ah, 00H
    int 16h          ; 执行后: AH=扫描码, AL=ASCII码

    ; 扫描码判断: ESC键 (扫描码=0x01) 则退出
    cmp ah, 0x01
    je exit_loop

    ; 回显输入字符
    mov ah, 09H
    mov bh, 0
    mov bl, 0x0E      ; 黄色属性
    mov cx, 1
    int 10h          ; 在当前光标位置输出字符

    call move_cursor_right

    ; 循环等待下一次按键
    jmp keyboard_loop

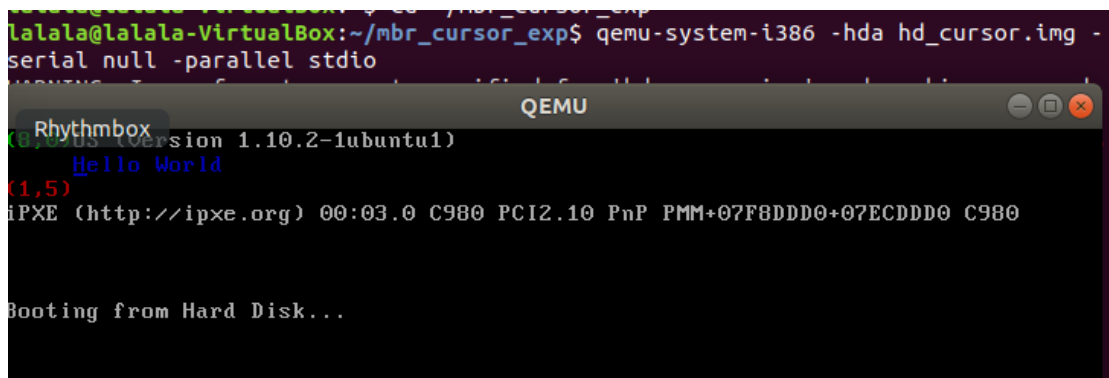
; 退出循环: 死循环保持界面
exit_loop:
    jmp $

```

● 实验结果展示:



- ① (0, 0) 是初始光标位置
- ② 在 (1, 5) 能看到青色的 Hello World
- ③ (1, 5) 移动后光标位置



```
lala@lala-VirtualBox:~/mbr_cursor_exp$ qemu-system-i386 -hda hd_number.img -serial null -parallel stdio
WARNING: Image format was not specified for 'hd_number.img' and probing guessed
帮助 QEMU
(8,0)OS (version 1.10.2-1ubuntu1)
24325242
```

▲输入“lalala”并成功显示，按 ESC 后界面不再响应按键

```
512 bytes copied, 0.000124148 s, 4.1 MB/s
lala@lala-VirtualBox:~/mbr_keyboard_scan$ qemu-system-i386 -hda hd_keyboard_scan.img -boot order=c -serial null -parallel stdio
WARNING: Image format was not specified for 'hd_keyboard_scan.img' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
QEMU
SeaBIOS (version 1.10.2-1ubuntu1)
IPXE (hlalalaipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+07F8DDDD+07ECDDDD C980
```

----- 思考题 17 -----

- 任务要求：汇编代码的编写，实现 your_if、your_while 和 your_function
- 思路分析：

1. 函数调用需要栈帧

```
asm ^
pushl %ebp ; 保存调用方的ebp
movl %esp, %ebp ; 建立当前函数的栈帧基址

结尾都有：
asm ^
popl %ebp ; 恢复调用方的ebp
ret ; 返回调用方
```

2. 注意除法的溢出风险以及系统调用参数错误（print_a_char 函数），否则可能出现段错误（常是由数组越界、空指针解引用或栈不平衡引起的）

```
lala@lala-VirtualBox:~/assignment$ make run
gcc -m32 -Wall -x assembler -c -o student.o student.asm
g++ -m32 -Wall -c -o test.o test.cpp
g++ -m32 -Wall -o student_test student.o test.o
./student_test
=== 分支逻辑结果 ===
a1 = 15, if_flag = 135
Makefile:24: recipe for target 'run' failed
make: *** [run] 段错误 (core dumped)
```

3.出现以下问题是由于缓冲区不同步，C++会将内容先存在一个内存缓冲区里。只有当缓冲区满了、程序结束、或者遇到 \n（换行符）时，才会统一交给系统去打印，在编写 test.cpp 时最开始没有加\n

```
=== 函数执行结果 ===
Hello Linux Assembly!string遍历输出:
```

4.注意寄存器污染问题

- 实验步骤:

①安装相应环境：在 lab1 已完成配置

②编写汇编代码

▲your_if

```

1  your_if:
2      movl a1, %eax          # 读取a1到32位寄存器eax
3
4      # 条件1: a1 < 12
5      cmpl $12, %eax
6      jge else_if_24        #如果大于等于就跳转
7
8      # 满足a1 < 12: if_flag = a1/2 +1
9      movl %eax, %edx
10     shrl $1, %edx          # a1/2
11     addl $1, %edx          # +1
12     movl %edx, if_flag
13     jmp if_end
14
15 else_if_24:
16     # 条件2: a1 < 24
17     cmpl $24, %eax
18     jge else_block
19
20     # 满足a1 <24: if_flag = (24 -a1)*a1
21     movl $24, %edx
22     subl %eax, %edx        # 24 - a1 (sub 源, 目的)
23     imull %eax, %edx
24     movl %edx, if_flag
25     jmp if_end
26
27 else_block:
28     # else分支: if_flag = a1 <<4
29     movl %eax, %edx
30     shll $4, %edx
31     movl %edx, if_flag
32
33 if_end:
34     ret
35

```

▲your_while

```

1  your_while:
2      pushl %ebp          #保存旧寄存器的值
3      movl %esp, %ebp
4
5  while_loop:
6      #条件: a2>=12
7      movl a2, %eax
8      cmpl $12, %eax
9      jl while_end
10
11     pushl %eax          # 调用前保存当前a2的值, 避免被函数破坏
12     call my_random      #调用my_random函数
13     movl %eax, %ecx     # 随机数存入ecx
14     popl %eax           # 恢复a2的值到eax
15
16     # 索引计算: a2 - 12
17     subl $12, %eax
18     js while_end        # 防止越界js=jump if sign,如果结果为负, 退出
19     cmpl $11, %eax
20     jg while_end
21
22     movl %ecx, while_flag(,%eax,4) # 寻址公式: while_flag的基地址 + eax*4
23
24     decl a2             # 直接让内存a2-1, decl=decrement long
25     jmp while_loop
26
27 while_end:
28     popl %ebp
29     ret

```

▲your_function

```

1  your_function:
2      pushl %ebp
3      movl %esp, %ebp
4      movl $0, %esi      # i=0, esi作为循环计数器
5
6  for_loop:
7      # 读取string[i]: string + esi (内存地址=基址(,%索引寄存器,1))
8      movb string(%esi), %al
9      cmpb $0, %al      # 判断是否为'\0'
10     je for_end
11
12     # pushal: 保存所有32位通用寄存器
13     # 调用print_a_char会修改寄存器, 需要先保存现场, 避免覆盖循环计数器esi等数据
14     pushal
15     # 将%al中的单字符零扩展为32位 (movzbl=move zero extend byte to long)
16     movzbl %al, %eax
17     pushl %eax
18
19     call print_a_char
20     addl $4, %esp      # 平衡栈 (弹出压入的四字节参数, 避免栈溢出)
21     # popal: 恢复所有寄存器
22     popal
23
24     incl %esi          # ++i
25     jmp for_loop
26
27 for_end:
28     movl %ebp, %esp
29     popl %ebp
30     ret

```

③此外还需要编写系统预定义函数实现

▲print_a_char:

最开始在 print_a_char 中使用了 lea 8(%ebp), %ecx。

- 8(%ebp) 是函数参数所在的**栈地址**。
- write 系统调用需要的是字符的**内存地址**。
- 如果 your_function 循环处理时栈帧被破坏, 或者 ecx 指向了不可读的内存区域, 就会崩溃, 出现段错误

```

1 # 打印单个字符
2 .section .data
3 char_buf: .byte 0      # 定义一个单字节缓冲区，直接引用栈地址作为write的缓冲区
4                        # 有时会不稳定
5
6 .section .text
7 print_a_char:
8     pushl %ebp
9     movl %esp, %ebp
10    pushal              # 保护所有寄存器
11
12    movl 8(%ebp), %eax  # 获取传入的字符
13    movb %al, char_buf
14
15    # 调用Linux 32位sys_write系统调用打印字符
16    movl $4, %eax      # eax=4, sys_write
17    movl $1, %ebx      # ebx=1, 指定输出目标为标准输出stdout (终端)
18    movl $char_buf, %ecx # ecx=char_buf地址: 指定要打印的数据内存地址
19    movl $1, %edx      # edx=1: 指定打印长度为1字节 (单个字符)
20    int $0x80          ## 触发软中断，执行系统调用 (将字符打印到终端)
21
22    popal              # 恢复所有寄存器
23    popl %ebp
24    ret

```

▲my_random

```

1 # 生成随机数
2 my_random:
3     pushl %ebp          # 保存调用方的栈帧基址
4     movl %esp, %ebp     # 将当前栈顶 (esp) 赋值给ebp, 固定本函数的栈帧基址
5
6     rdtsc              # 读取时间戳计数器, 低32位在eax, 高32位在edx
7     andl $0xFFF, %eax   # 取低12位, 限制随机数范围
8     popl %ebp          # 回复调用方的ebp寄存器
9     ret

```

④编写 test.cpp 来测试汇编代码 (详见附件)

⑤编写 Makefile 来自动化编译、链接和运行一个 32 位程序 (详见附件)

● 实验结果展示:

```

lalala@lalala-VirtualBox:~/assignment$ make run
gcc -m32 -Wall -x assembler -c -o student.o student.asm
g++ -m32 -Wall -c -o test.o test.cpp
g++ -m32 -o student_test student.o test.o
./student_test
=== 分支逻辑结果 ===
a1 = 15, if_flag = 135

=== 循环逻辑结果 ===
a2初始值=15, while_flag数组结果:
while_flag[0] = 2762
while_flag[1] = 2608
while_flag[2] = 2375
while_flag[3] = 2114

=== 函数执行结果 ===
string遍历输出:
Hello Linux Assembly!
lalala@lalala-VirtualBox:~/assignment$

```

----- 思考题 14-----

- 任务要求：请编写一个字符弹射程序，其从点（2，0）处开始向右下角 45 度射出，遇到边界反弹，反弹后按 45 度角射出，方向视反弹位置而定（加入变色效果）
- 思路分析：
 - 1.程序应该不超过 510 字节，否则无法放入 MBR 中被加载执行
 - 2.注意 CPU 不允许混搭不同位宽的操作数（add 指令），否则会出现如下报错

mbr.asm:56: error: invalid combination of opcode and operands

- 实验步骤：
 - ①编写汇编代码

```

1  [bits 16]    ;十六位实模式汇编代码
2  [org 0x7c00] ;程序加载到的内存地址
3
4  start:
5      ; 设置 80x25 文本模式
6      mov ax, 0x0003
7      int 0x10
8
9      mov dx, 0x0002
10     mov bl, 1
11
12     mov cl, 1          ; X 方向步长
13     mov ch, 1
14
15 draw:
16     ; 1. 设置光标位置
17     push dx             ; 保存当前坐标
18     mov ah, 0x02        ; 设置光标位置
19     xor bh, bh          ; 指定页面0
20     int 0x10
21
22     ; 2. 打印带颜色的字符 'O'
23     mov ax, 0x094f      ; AH=0x09 (写字符和属性), AL=0x4f (ASCII码的'O')
24     inc bl              ; 变色: 颜色自增
25     ; 避开黑色, 确保字符可见
26     test bl, 0x0F
27     jnz .skip_black
28     inc bl
29 .skip_black:
30     mov cx, 1           ; 只打印一个字符
31     int 0x10
32     pop dx
33
34     ; 3. 延时 (约 0.05 秒)
35     mov cx, 0x0000
36     mov di, 0x8000      ; 延时计数器
37     mov ah, 0x86        ; AH下, 精确延时
38     int 0x15
39
40     ; 4. 擦除旧字符
41     mov ah, 0x02
42     xor bh, bh
43     int 0x10
44     mov ax, 0x0920      ; 空格, 空格覆盖, 实现擦除
45     mov cx, 1
46     int 0x10
47
48     ; 5. 更新坐标与碰撞检测
49     ; 找回方向值, 因为 cx 被 int 10h 污染了
50
51     ; 处理 X 轴
52     add dl, byte [x_dir]
53     cmp dl, 0           ; 检查是否到达屏幕左边界
54     jle .flip_x         ; 是则反方向
55     cmp dl, 79          ; 检查是否到达屏幕右边界
56     jge .flip_x
57     jmp .check_y
58 .flip_x:
59     neg byte [x_dir]
60     add dl, [x_dir]
61     add dl, [x_dir]      ; 两次相加: 抵消之前的越界移动 + 反向移动
62
63 .check_y:
64     ; 处理 Y 轴
65     add dh, [y_dir]
66     cmp dh, 0
67     jle .flip_y
68     cmp dh, 24
69     jge .flip_y
70     jmp draw
71 .flip_y:
72     neg byte [y_dir]
73     add dh, [y_dir]
74     add dh, [y_dir]
75     jmp draw
76
77 ; 数据区
78 x_dir db 1
79 y_dir db 1
80
81 ; 填充
82 times 510-($-$$) db 0 ; 表示填充字符0直到第510个字节
83 dw 0xaa55             ; 引导扇区标志 (必须以0xaa55结尾, BIOS才会识别为可引导)

```

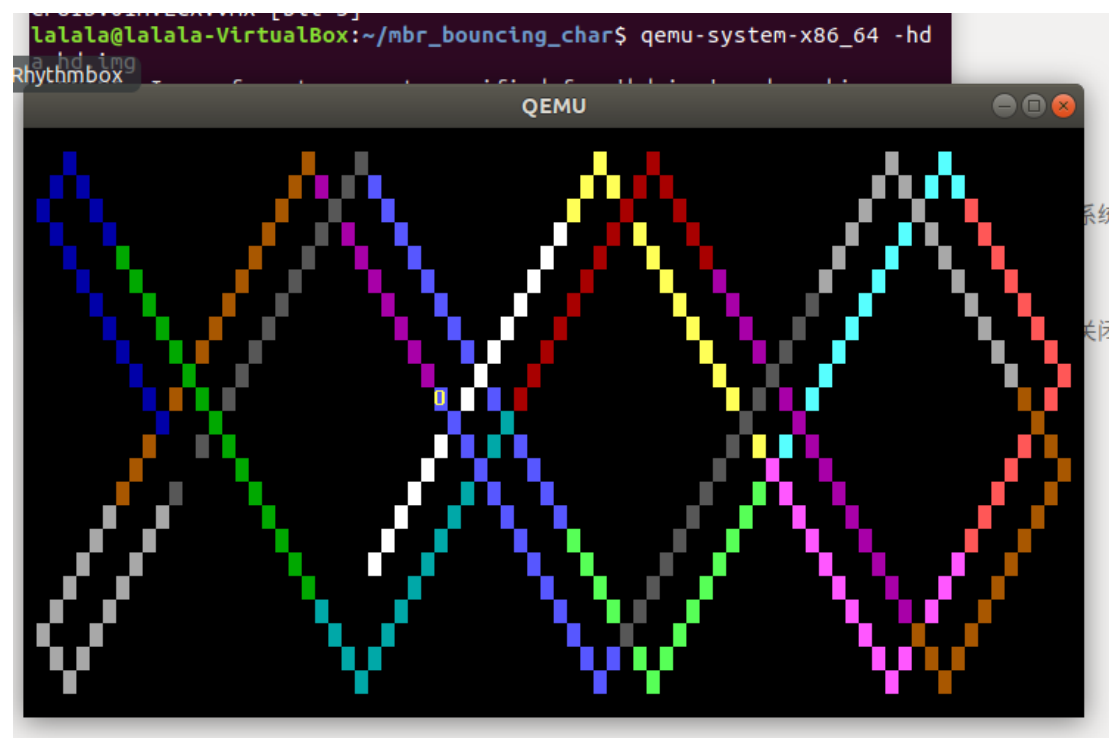
②使用 NASM 将代码编译成纯二进制文件

重复前面 hello world 步骤：

③创建虚拟硬盘镜像并将 MBR 写入首扇区

④启动虚拟机测试

● 实验结果分析：



Section 5 实验总结与心得体会

1. 通过本次实验学习到 x86 汇编、计算机的启动过程、IA-32 处理器架构（特别是实模式操作模式）和字符显存原理。对计算机的启动有了更深入的理解
2. 注意除法的溢出风险以及系统调用参数错误，否则可能出现段错误（常是由数组越界、空指针解引用或栈不平衡引起的）
3. C++会将内容先存在一个内存缓冲区里。只有当缓冲区满了、程序结束、或者遇到 `\n`（换行符）时，才会统一交给系统去
4. CPU 不允许混搭不同位宽的操作数（add 指令），否则会出现报错

Section 6 附录：参考资料清单

实验指导——[SYSU-2023-Spring-Operating-System: 中山大学2023学年春季操作系统课程 - Gitee.com](#)